

Keep the useful play working

Opt-out

→ Exclude the recipient

Human reply

→ Stop cold outreach and review the conversation

Active booking

→ Stop cold outreach and prepare the meeting

Unknown provider result

→ Read the existing attempt

Read replies before any cold follow-up

12 min / Easy / Connected own Gmail and scoped thread history. A real reply is optional; drafting is local.

- 1 Run the reply read. Use the sender's actual native thread.
- 2 Inspect the classification. Separate human, automatic, bounce and opt-out.
- 3 Prepare the next answer. Keep it in the existing conversation.
- 4 Save the stop decision. Cold outreach must not conflict with the reply.

You should have: Native reply classification, stop decision and reply draft where useful.

Check: The reply is bound to the exact earlier thread.; Opt-outs and human replies exclude cold outreach.; Drafting is not counted as sending.

Full prompt on the next page and at prompts.html#p3-replies

Prompt: Read replies before any cold follow-up

If the required connection or initialized runtime is unavailable, save that exact gap in company-motion/REPLIES.md and name the next setup task. Do not claim the read or cycle completed; keep the finished local page play intact. Show the gap and next setup task, then stop here. Otherwise continue below.

Refresh and read the actual scoped mailbox/thread history for our original test and pilot.

Save company-motion/REPLIES.md with each exact thread ID, observed reply or no-reply result and next action.

Confirm any reply, opt-out or relationship excludes further cold follow-up. Use the current runtime's actual checks.

For a real reply, draft a short response from its words and our sourced offer. Keep it local for review; automatic reply sending is not wired.

If no reply exists, say so. Use a clearly labeled local fixture only to inspect stop logic. Never count that fixture as a real response.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop.

Next: read actual bookings and cancellations.

Copy full prompt: [prompts.html#p3-replies](#)

Next: read actual bookings and cancellations (p3-bookings)

Read the actual booking record

10 min / Easy / Google Calendar. Calendly token and organization/event scope only if used.

- 1 Read the configured booking source. Use your own organization and event scope.
- 2 Match the attendee and company. Save the actual native meeting ID.
- 3 Check a reschedule or cancellation. Keep the replacement meeting active.
- 4 Inspect the cold-outreach exclusion. An active booking changes the next action.

You should have: BOOKINGS.md with actual booking states or a successful empty read.

Check: Actual booking IDs and state are saved when present.; An empty read and attendance are labeled separately.

Full prompt on the next page and at [prompts.html#p3-bookings](#)

Prompt: Read the actual booking record

If the required connection or initialized runtime is unavailable, save that exact gap in company-motion/BOOKINGS.md and name the next setup task. Do not claim the read or cycle completed; keep the finished local page play intact. Show the gap and next setup task, then stop here. Otherwise continue below.

Read bookings from my configured owned calendar or Calendly source.

Save company-motion/BOOKINGS.md with actual source IDs, start times and current states. A successful empty read is zero bookings, not a connection failure.

For an actual reschedule, preserve the old record and active replacement. Verify an active booking holds cold follow-ups.

If I want a controlled calendar test, show the exact owned test event for approval before creating it, label it as a test and remove it only with my approval.

Do not infer attendance from a booking. Automatic attendance attribution is not wired.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop.

Next: inspect your complete working play.

Copy full prompt: [prompts.html#p3-bookings](#)

Next: inspect your complete working play (p3-working-play)

Chapter 2 of 5

Check one complete play

Evidence from company facts through reviewed output and actual result.

17 guided minutes

Open the next full prompt. Review the result before moving on.

Show your first working play from start to finish

12 min / Easy / Saved company/target files and actual provider records. No new channel.

- 1 Open one completed play. Read its target facts, output and review together.
- 2 Trace its actual result. Show file evidence or the exact provider record.
- 3 Repair one broken local handoff. Preserve committed actions and rerun the check.

You should have: WORKING-PLAY.md with a complete evidence chain.

Check: Each handoff has an actual file or provider ID.; Unsupported claims and missing stages remain visible.

Full prompt on the next page and at prompts.html#p3-working-play

Prompt: Show your first working play from start to finish

Choose our completed account-page or email play.

Open the actual target brief, saved skill, reviewed output and result together.

For a page, trace target fact to page claim to source review. For email, trace current buyer evidence to exact approved message to original attempt to Gmail Sent match.

Repair one unsupported claim or broken handoff and rerun its check without replacing a committed action.

Save company-motion/WORKING-PLAY.md with actual paths/IDs, passed checks, exact stage and one next action.

Label local-only and live outcomes separately.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop.

Next: take the five-minute break.

Copy full prompt: [prompts.html#p3-working-play](#)

Next: take the five-minute break (p3-break)

Five minutes. Save the channel results.

Save your working-play record. Leave the same project open. Return in five minutes.

A gift can wait for a recipient. Your notes should not.

Chapter 3 of 5

Repeat without duplicating work

Two bounded cycles, retained history and pause/reopen proof.

20 guided minutes

Open the next full prompt. Review the result before moving on.

Run two preparation cycles without repeating actions

12 min / Easy / Same project, supported reads and existing exact grants. No new send required.

- 1 Run a paused preparation cycle. Read history and current sources before choosing work.
- 2 Repeat from the same project. The second cycle reads the first record.
- 3 Compare the actual decisions. Earlier actions stay unchanged; new copy stays a draft.

You should have: MANUAL-CYCLES.md with two actual timestamps and observed decisions.

Check: Two distinct manual cycles are observed.; Original action keys and attempt counts remain intact.; New copy stays local until separately approved.

Full prompt on the next page and at prompts.html#p3-manual-cycles

Prompt: Run two preparation cycles without repeating actions

If the required connection or initialized runtime is unavailable, save that exact gap in company-motion/MANUAL-CYCLES.md and name the next setup task. Do not claim the read or cycle completed; keep the finished local page play intact. Show the gap and next setup task, then stop here. Otherwise continue below. Run two manual preparation/refresh/review cycles from this same project with new sending paused. Each cycle reads saved history, original pending IDs, actual exclusions, scoped mailbox history and bookings before selecting a next action. Use the supported read commands and save actual timestamps, output and decisions in company-motion/MANUAL-CYCLES.md. The second cycle must keep earlier sent/unknown actions unchanged. Do not resubmit or reimport them. New copy remains a draft without a new exact grant. Show both cycles and verify the saved attempt counts and original keys. Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: pause and reopen the same project.

Copy full prompt: [prompts.html#p3-manual-cycles](#)

Next: pause and reopen the same project (p3-pause)

Pause. Reopen. Keep the history.

8 min / Easy / Same local project and execution controls.

- 1 Pause with the documented command. Read the actual local paused state.
- 2 Reopen the same project. Inspect the original records and history.
- 3 Run a read-only review. It must not make a new recipient-facing commitment.
- 4 Keep sending paused. Provider queues need separately approved native controls.

You should have: Pause and recovery evidence with retained history.

Check: Paused local execution cannot make a new commitment.; Provider-side pending work is checked separately.; Restart retains attempts and exclusions.; Unknown outcomes cause readback rather than another send.

[Full prompt on the next page and at `prompts.html#p3-pause`](#)

Prompt: Pause. Reopen. Keep the history.

If the required connection or initialized runtime is unavailable, save that exact gap in company-motion/RECOVERY.md and name the next setup task. Do not claim the read or cycle completed; keep the finished local page play intact. Show the gap and next setup task, then stop here. Otherwise continue below.

Run pause and status through the saved project launcher. Confirm the actual local paused state.

Reopen the same project and inspect the saved database, original action keys, attempts, exclusions and pending provider IDs.

Run a read-only preparation/review cycle while paused; do not make a new recipient-facing commitment.

Save company-motion/RECOVERY.md with the pause result and retained-history evidence.

If a provider queue or gift is still active, read its original object and show the specific provider stop/cancel control for my approval. Local pause alone does not cancel it.

Keep new sending paused at the end of this exercise.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop.

Next: check whether unattended operation is ready.

Copy full prompt: [prompts.html#p3-pause](#)

Next: check whether unattended operation is ready (p3-schedule)

Chapter 4 of 5

Prepare a schedule when ready

Readiness record. Schedule remains paused unless every gate passes.

20 guided minutes

Open the next full prompt. Review the result before moving on.

Prepare a schedule. Activate only when ready.

10 min / Moderate / Supported local automation feature; complete native sources only for unattended operation.

- 1 Read both manual cycles.
- 2 Check unattended prerequisites.
- 3 Prepare a paused schedule.
- 4 Review before activation.

You should have: SCHEDULE-READINESS.md and a paused schedule only when the app supports it.

Check: No incomplete source is silently replaced by an old snapshot.; Actual scheduling capability is checked.; No schedule becomes active without exact recurring approval.

Full prompt on the next page and at prompts.html#p3-schedule

Prompt: Prepare a schedule. Activate only when ready.

Read both actual manual cycles and the unattended-operation guide.

Save company-motion/SCHEDULE-READINESS.md. Check complete fresh native customer, CRM, scoped/full configured mailbox history and booking reads, plus exact ongoing owner authority. Snapshot mode cannot support unattended operation.

If any gate is missing, name its exact setup task and keep our working play manual.

If ready and a real local scheduling tool is available, prepare one paused schedule for this same project. Show actual workspace, timezone, wake instructions and first two scheduled times.

Each wake must read history and sources, prepare exact work, and release only separately approved keys. New copy is a draft until approved. Keep the Mac awake and online.

Do not activate a schedule from this prompt. Show it for my exact approval.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop.

Next: record actual scheduled observations or leave them pending.

Copy full prompt: [prompts.html#p3-schedule](#)

Next: record actual scheduled observations or leave them pending (p3-two-cycles)

Count observed runs, not planned runs

10 min / Moderate / Actual manual cycle records; optional active schedule only after separate approval.

- 1 Open both manual cycle records. Check their actual timestamps and decisions.
- 2 Inspect any supported schedule. Leave it paused unless separately approved.
- 3 Save actual observations. Unobserved scheduled wakes remain pending.
- 4 Keep the original action history. No earlier action is resubmitted.

You should have: CYCLE-OBSERVATIONS.md with manual and scheduled outcomes labeled separately.

Check: Two manual cycles have real timestamps.; Unobserved scheduled wakes stay pending.; Earlier actions are not resubmitted.

Full prompt on the next page and at prompts.html#p3-two-cycles

Prompt: Count observed runs, not planned runs

Read company-motion/MANUAL-CYCLES.md and any actual supported schedule record.
Save company-motion/CYCLE-OBSERVATIONS.md distinguishing observed manual cycles from scheduled wakes.
If a schedule is still paused or missing exact recurring approval, leave it paused. Record that stage honestly.
If separately approved and active, observe only wakes that actually occur. Save their exact times, original action IDs and decisions; check no earlier action was resubmitted.
For a wake outside class, save its real next observation time in company-motion/WAITS.md. Do not manufacture a completed observation.
Keep pending provider outcomes tied to their original IDs.
Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop.
Next: record what you have actually built and what is still missing.

Copy full prompt: [prompts.html#p3-two-cycles](#)

Next: record what you have actually built and what is still missing (p3-checklist)

The framework supports more than today's one play

Email

Guarded first email and exact Gmail Sent match

Gifts

Bounded Loop & Tie creation and exact provider stage

Bookings

Read configured sources. Attendance association is not wired.

Advertising

Source: creative. Portable: approved drafts/readback. Launch/restart CLI held; owned UI review needed.

LinkedIn

Owned UI campaign setup; one exact paused-contact release and provider queue/sent read

Replies

Read/classify; Codex drafts. Automatic reply sending is not wired.

Pages

Codex builds; portable site/owner/bytes-bound publication.

Unattended operation

Complete provider sources and existing exact grants. New copy waits for owner approval.

Chapter 5 of 5

Record results and choose the next play

Honest completion record. Extra channels are optional guides.

8 guided minutes + 30 minutes for help

Open the next full prompt. Review the result before moving on.

Your framework, working play and remaining gaps

8 min / Easy / Codex and your workshop project.

- 1 Open actual files and provider records. Use their IDs, dates and precise stages.
- 2 Save COMPLETION.md. Mark each capability from observed evidence.
- 3 Update WAITS.md. Keep original IDs, next reads and missing setup.

You should have: Evidence-linked full capability record.

Check: Core local completion is assessed separately from conditional email and operation.; Each result has actual evidence and a date.; Missing access stays a gap; scheduling stays optional.

Full prompt on the next page and at [prompts.html#p3-checklist](#)

Prompt: Your framework, working play and remaining gaps

Build company-motion/COMPLETION.md from actual files and checks.

First record core local completion: personal framework, company/target briefs, two reviewed pages, reused skill, paused authority/instructions, execution flow and prepared ten-company universe. Link each actual artifact and review result.

Separately record conditional email results: authenticated sender, buyer/validation evidence, exact self-test and prospect pilot stages. Record replies, bookings and provider cycles only when actually connected and observed.

Missing access is a named gap; it does not fail a completed local page play.

Record social, gifts, ads, public hosting and unattended operation as optional extensions. Keep automatic reply sending and attendance marked not wired. Never count queued, fixture or unobserved results as live.

Give every gap one next action and keep original pending IDs in company-motion/WAITS.md.

Append the actual check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: finish the first remaining issue with individual help.

Copy full prompt: [prompts.html#p3-checklist](#)

Next: finish the first remaining issue with individual help (p3-help)

Finish one blocked operation at a time

Work individually. Open your capability record. Repair the first connection or failed check. Keep completed work and history.

Prompt: Finish one blocked operation at a time

Read company-motion/COMPLETION.md and the first incomplete main-lane check.
Repair an available local cause, rerun the required check and record the actual result.
Preserve working outputs, original keys, commitments and limits.
If only an optional extension remains, name its specific account, resource, cost and approval setup. Do not claim that instructions alone reproduce a live channel.
Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop.
Next: continue with the same saved project after class.

Copy full prompt: [prompts.html#p3-help](#)

Next: continue with the same saved project after class (p3-take-home)

Keep your working project. Add one channel at a time.

You keep the prepared framework, your company context and a verified play. Add the next channel when its account, resources and exact approval are ready.

Company facts

Two reviewed pages

Reused page skill

Execution rules

Prepared target universe

Honest gap record

Next actions

Publish the page to your owned site

Optional after class / Moderate / Owned Netlify token, fresh workshop site and reviewed standalone HTML. Browser access for desktop/phone checks.

- 1 Show the reviewed HTML and destination. Use your own dedicated site.
- 2 Approve the exact publication. Check company, file and account.
- 3 Publish through the documented operation. Save the native deployment result.
- 4 Open desktop and phone views. Check the actual form or next-step link.

You should have: Native deployment result and verified public page.

Check: Returned approval binds owned site, owner and exact bytes.; Changed site or file is held before upload.; Actual HTTPS URL shows the reviewed bytes and readable layouts.; A form has separate submission evidence when used.

Full prompt on the next page and at [prompts.html#p3-publish](#)

Prompt: Publish the page to your owned site

Open prompts.html

Find "Publish the page to your owned site". Click Copy prompt.

Paste it into the local Claude Code or Codex project you opened for this workshop.

The copy source includes the full instructions. This slide contains no partial prompt.

Copy full prompt: [prompts.html#p3-publish](#)

Next: record this extension's actual stage in COMPLETION.md (p3-checklist)

LinkedIn: from tabs to a checked queue

Before

Find the profile

Send a connection

Copy a message

Track the conversation

With your workflow

Verify current identity

Prepare owned paused contact

Release approved stage once

Read native queue or sent event

Verify your LinkedIn seat, list and campaign

Optional after class / Moderate / Gojiberry paid account and owned LinkedIn seat/list/campaign.

- 1 Open your owned provider dashboard.
- 2 Read the exact IDs.
- 3 Prepare one paused contact.
- 4 Inspect every campaign list.

You should have: One verified paused contact and exact social review.

Check: Dedicated campaign/list/seat belong to your account.; Campaign has invitation steps only and remains inactive during preparation.; Exact created contact is paused with readyForCampaign:false.; All campaign lists contain only paused contacts.

Full prompt on the next page and at [prompts.html#p3-social-setup](#)

Prompt: Verify your LinkedIn seat, list and campaign

Open prompts.html

Find "Verify your LinkedIn seat, list and campaign". Click Copy prompt.

Paste it into the local Claude Code or Codex project you opened for this workshop.

The copy source includes the full instructions. This slide contains no partial prompt.

Copy full prompt: [prompts.html#p3-social-setup](#)

Next: record this extension's actual stage in COMPLETION.md (p3-checklist)

Release one approved LinkedIn stage

Optional after class / Moderate / Owned Gojiberry seat, isolated provider UI campaign, one paused test contact and exact one-stage approve-scope grant.

- 1 Approve the exact stage. Use your willing contact and owned seat.
- 2 Prepare one guarded action. Use the supported schema and command.
- 3 Run the exact approved key once. Use the action-key selector and a limit of one.
- 4 Read the native result. A message waits for real invitation acceptance.

You should have: One provider social result and action key.

Check: Exact grant allows one supported social stage only.; Provider contact and campaign match that approval.; Queued is not counted as sent.; Recipient waits keep the original action key.; The manual command selects the exact original approved social key, without releasing another grant.

Full prompt on the next page and at [prompts.html#p3-social-run](#)

Prompt: Release one approved LinkedIn stage

Open prompts.html

Find "Release one approved LinkedIn stage". Click Copy prompt.

Paste it into the local Claude Code or Codex project you opened for this workshop.

The copy source includes the full instructions. This slide contains no partial prompt.

Copy full prompt: [prompts.html#p3-social-run](#)

Next: record this extension's actual stage in COMPLETION.md (p3-checklist)

Gifting: one offer, exact terms, provider proof

Before

Choose the recipient, collection and terms.
Check the result by hand.

With your workflow

Prepare the exact offer. Check funding and meeting terms. Submit once. Read its provider stage.

Message preview

Status: Gift sent

Subject: a small thing from the Metadata team

Recipient: [redacted]

Amount: \$150

Loop & Tie reports this gift email was sent.

Exact message

We made a page for your team: <https://demo.metadata.io/datarails/>

Request a demo there and we will create a personalized commercial for your company. You will see the scheduling link after submitting the form.

This gift is an invitation to meet the Metadata team. Book a meeting to unlock your gift selection.

Actual sent message, recreated for readability; recipient redacted. Datarails, September 30. Gift value: \$150.

Check the collection, funding and meeting gate

Optional after class / Moderate / Loop & Tie account, funded owned team, collection and verified native meeting gate.

- 1 Read your gift team and balance. Check the real collection cost.
- 2 Verify the meeting gate. Read its native numeric ID and binding.
- 3 Write truthful terms. Include value, expiry and meeting requirement.
- 4 Preview the exact recipient. Use one approved gift recipient.

You should have: Verified gift prerequisites and exact gift preview.

Check: Actual team, catalogue and form identify the same owned scheduler.; External ID and enabled meeting-required input are inside the same numeric form.; Signed-in capture URL/team match and actual capture is within 65 minutes.; Collection cost/currency, value and expiry are truthful.

Full prompt on the next page and at prompts.html#p3-gift-setup

Prompt: Check the collection, funding and meeting gate

Open prompts.html

Find "Check the collection, funding and meeting gate". Click Copy prompt.

Paste it into the local Claude Code or Codex project you opened for this workshop.

The copy source includes the full instructions. This slide contains no partial prompt.

Copy full prompt: [prompts.html#p3-gift-setup](#)

Next: record this extension's actual stage in COMPLETION.md (p3-checklist)

Create one approved gift. Read its stage.

Optional after class / Moderate / Funded Loop & Tie team and approved one-gift scope. Real collection charge may apply.

- 1 Approve the exact gift and cost. Use the saved preview.
- 2 Run the exact approved gift key once. Use the action-key selector and a limit of one.
- 3 Read the native gift. Match recipient, collection, copy and expiry.
- 4 Save the provider stage. Created, sent and redeemed differ.

You should have: One provider gift ID and exact provider stage.

Check: Immutable grant names this exact recipient, terms and key.; Actual collection price/currency fit the per-gift and lifetime ceilings.; Only one gift is authorized.; Prior gifts and unknown commitments are not repeated.; The manual command selects the exact original approved gift key, without releasing another grant.

Full prompt on the next page and at [prompts.html#p3-gift-run](#)

Prompt: Create one approved gift. Read its stage.

Open prompts.html

Find "Create one approved gift. Read its stage.". Click Copy prompt.

Paste it into the local Claude Code or Codex project you opened for this workshop.

The copy source includes the full instructions. This slide contains no partial prompt.

Copy full prompt: [prompts.html#p3-gift-run](#)

Next: record this extension's actual stage in COMPLETION.md (p3-checklist)

Recorded MQLs: one to two. Pipeline unverified.

	Prior period	Latest period
Recorded MQLs	1	2
Pipeline impact: unverified.		
Spend	\$9,229	\$51,867
Impressions	36,627	511,266
Clicks	254	2,966
Leads	4	43

Audit the 43 leads, meeting outcomes and opportunity attribution before raising total spend.

August 1-30 versus August 31-September 29, 2026. Complete UTC days.

Turn your ad results into one clear action

Optional after class / Easy / Own Metadata/advertising connection or official ad export. Creative credits only if approved generation is used.

- 1 Read your own ad account. Use a connected Metadata account or an official export.
- 2 Compare complete equal periods. Keep spend, clicks, leads and qualification separate.
- 3 Save three concrete improvements. Use the actual experiment evidence.
- 4 Prepare one creative brief. Keep launch and budget changes for separate approval.

You should have: Real ad comparison, three recommendations and one reviewed creative brief.

Check: Periods and denominators match.; No pipeline attribution is invented.; Generated creative is not called a live campaign.

Full prompt on the next page and at prompts.html#p3-ad-review

Prompt: Turn your ad results into one clear action

Open prompts.html

Find "Turn your ad results into one clear action". Click Copy prompt.

Paste it into the local Claude Code or Codex project you opened for this workshop.

The copy source includes the full instructions. This slide contains no partial prompt.

Copy full prompt: [prompts.html#p3-ad-review](#)

Next: record this extension's actual stage in COMPLETION.md (p3-checklist)

Build one ad draft. Review it in Metadata.

Optional after class / Moderate / Own Metadata account/token, reviewed dedicated campaign, native readiness and separate approved advertising budget. Real spend may apply.

- 1 Read your real account and budget group. Use owned resources and actual spending ceilings.
- 2 Review the exact draft request. Audience, offer and creative must match your approval.
- 3 Create only the approved draft. Read its provider ID and non-running state.
- 4 Inspect it in Metadata. A later launch requires separate exact approval.

You should have: Exact approved draft receipt, owned Metadata UI review and actual campaign state.

Check: Owned account, audience and budget group match the review.; Draft creation cannot start spend under the inspected provider contract.; Daily and total budgets fit the separate owner authority.; Launch/restart CLI remains held.; Draft is not labeled a running campaign.

Full prompt on the next page and at [prompts.html#p3-ad-test](#)

Prompt: Build one ad draft. Review it in Metadata.

Open prompts.html

Find "Build one ad draft. Review it in Metadata.". Click Copy prompt.

Paste it into the local Claude Code or Codex project you opened for this workshop.

The copy source includes the full instructions. This slide contains no partial prompt.

Copy full prompt: [prompts.html#p3-ad-test](#)

Next: record this extension's actual stage in COMPLETION.md (p3-checklist)